

Student hands-on workbook: RAG with evidence and guardrails

English · Student hands-on workbook

Author

Rajkumar Rajagobalan

Use this workbook alongside the executable Colab notebook. Follow the link in the opening section. Record observations on paper or in your saved notebook.

Name or pair ID: _____ Date: _____

[Open the step-by-step workbook in Colab](#)

This workbook accompanies the executable Colab notebook. Read the concept, predict what will happen, run the experiment, and explain the result. The reference code is supplied so you can spend the session investigating behavior rather than typing infrastructure code.

You will build a question-answering assistant for a **fictional university library**. The ten policies are synthetic classroom data. They include a current renewal rule and an archived, conflicting rule so you can observe a realistic evidence-selection problem.

Learning outcomes

By the end, you should be able to:

- Explain how chunk boundaries and overlap affect the evidence available to a retriever.
- Compare lexical and dense retrieval using ranked source documents.
- Generate an answer with traceable evidence and distinguish provenance checks from factual support.
- Test input limits, fabricated citations, unsupported questions and a prompt-injection attempt.
- Calculate retrieval metrics, inspect generation separately, and propose a useful next test.

Your session map

Class clock	Activity	Evidence to save
00–10	Setup and concepts	Successful preflight
10–30	Lab 1: chunking and metadata	Two chunking configurations
30–55	Lab 2: retrieval and filtering	Rankings and filter comparison
55–60	Break and save	Saved notebook
60–85	Lab 3: grounded answers and guardrails	Answer, citations and rejection results
85–110	Lab 4: evaluation and adversarial testing	Metric comparison and attack review
110–120	Submission and discussion	Notebook and JSON report

How to record your work

Use the tables below as worksheets. In Colab, either edit the relevant text cell or insert a **Text** cell underneath an experiment to record observations. At the end of each lab, also complete its `reflections['labN']` string in the supplied code cell. These strings are what the JSON submission exports.

For example, replace an empty reflection with your own evidence:

```
reflections['lab1'] = "My two counts were ... . The repeated phrase was ... . This matters because ... ."
```

Do not submit the example sentence. Write your measured values and interpretation. After editing a reflection, run that cell so the value exists in the runtime. Re-running a cell that still contains `''` will reset its reflection to empty. Save useful output before rerunning experiments because Colab replaces that cell's previous output.

Preparation and core concepts

Time: 00–10 minutes. Complete setup while the instructor introduces the pipeline.

Concept: what RAG adds

Retrieval-augmented generation supplies relevant source text to a generative model when a question arrives. The source corpus provides facts that may be private, recent, or specific to the application. The model still has to interpret that evidence correctly.

There are two flows in this notebook:

Flow	What happens	Main notebook objects
Preparation	Load records, split text, embed chunks and build an index	<code>DOCUMENTS</code> , <code>chunks</code> , <code>dense_matrix</code>
Question answering	Rank evidence, apply eligibility, build the request, generate and check output	<code>search</code> , <code>answer_question</code> , <code>generate_grounded</code> , <code>validate_output</code>

Retrieval can fail by selecting the wrong evidence. Generation can fail even when retrieval found the right evidence. Keep these failure types separate throughout the workbook.

Step 0.1 — Open and save your notebook

1. Open the Colab link at the top of this workbook and sign into your Google account.
2. Select **Copy to Drive** to create your own copy.
3. Rename it `RAG_Workbook_<your-name-or-pair-id>`.
4. In the runtime settings, use **Python 3** and **None / CPU** as the hardware accelerator.
5. Open the notebook's table of contents. Locate Setup A, Setup B and Labs 1–4.

Check: you have your own editable copy and can locate all four lab sections.

Step 0.2 — Configure your OpenAI key

1. Open Colab's **Secrets** panel.
2. Add a secret named exactly `OPENAI_API_KEY`.
3. Enter your funded OpenAI API key and enable notebook access for it.
4. Run the code cell under **Setup A · packages and secret**.

The key is used in the request's authorization header. It is not part of the model prompt. Keep it out of notebook text, code, screenshots and submissions.

Step 0.3 — Check both API capabilities

1. Read **Setup B · API client and embedding cache**.
2. Run its code cell once.
3. Check for the `READY` message. Setup tests both the embedding and generation endpoints.
4. Note that `api_calls` counts attempted requests and `usage_log` stores usage from successful requests. The embedding cache is keyed by model ID and exact text.

Concept: a preflight catches account or model-access problems before a later lab depends on them. A cache avoids sending unchanged text for embedding again during this runtime. The 40-request classroom limit is not an account spending cap; rerunning setup resets the counter and cache.

If blocked: check the secret for 401, model/project access for 403 or 404, and quota for 429. After two unsuccessful setup attempts, ask the instructor to pair you with a working student. Do not describe a recorded instructor response as your own live result.

Setup record: READY observed: _____ Runtime: _____ Pair ID, if applicable: _____

Lab 1 · evidence and chunk boundaries

Time: 10–30 minutes · **Chapters** 1–2

Your task: split source documents into chunks while retaining provenance, then investigate what overlap changes. Allocate 3 minutes to the walkthrough, 12 to experiments and 5 to recording and discussion.

Concepts before you code

Concept	Meaning in this lab	Why it matters
Document	A complete library policy record	Establishes the original source
Chunk	A smaller text window from that record	Determines which evidence can be retrieved together

Concept	Meaning in this lab	Why it matters
Overlap	Words repeated between adjacent windows	May preserve evidence across a boundary, but duplicates text
Metadata	<code>doc_id</code> , <code>title</code> and <code>status</code>	Supports provenance and eligibility decisions
Chunk ID	Source ID plus starting word offset	Identifies the particular passage you retrieved

This implementation splits on whitespace. Its sizes count **words, not model tokens**. It can cut sentences and separate a rule from an exception. You will inspect that limitation directly.

Step 1.1 — Inspect the source records

1. Run the cell beginning `DOCUMENTS = [...]`.
2. Confirm that it prints ten records.
3. Find D02 and D03 in the data. Read their text and status.
4. Identify the current renewal rule and the archived rule. Predict what could happen if the archived record reaches the generator.

Write: current source ID: _____ archived source ID: _____ conflicting durations: _____

Step 1.2 — Trace the chunking function

Read `chunk_documents`. Explain these operations to your partner before executing it:

- `size - overlap` determines the stride between window starts.
- `words[start:start + size]` bounds the current window.
- `{**doc, ...}` copies source metadata into each chunk.
- The final break stops once a window reaches the document end.

Predict: for a size of 20 and overlap of 5, the next window starts _____ words after the current one.

Step 1.3 — Run without overlap

1. In the cell containing `chunk_documents`, set `CHUNK_SIZE = 20` and `OVERLAP = 0`.
2. Run that cell.
3. Record `Trial chunks` and read all printed D02 chunks.
4. Underline or copy one sentence fragment at a boundary. Check whether a single chunk contains both the renewal duration and the complete reservation exception.

Step 1.4 — Change one parameter

1. Keep `CHUNK_SIZE = 20` and change only `OVERLAP` to 5.
2. Rerun the same cell.

- Record the new count and one repeated phrase.
- Compare evidence coverage. If overlap still fails to keep the full rule together, record that result rather than assuming it solved the problem.

Configuration	Total chunks	Boundary or repeated phrase	Rule and exception together?
20 words, overlap 0	_____	_____	_____
20 words, overlap 5	_____	_____	_____

Step 1.5 — Save your finding and prepare the shared corpus

- Complete `reflections['lab1']` with both counts, a quoted boundary observation and your interpretation.
- Run that reflection cell. It also creates the shared `chunks` corpus using **40 words / 8 overlap**.
- Keep this shared configuration for Labs 2–4. Your experimental `trial` is separate from the retrieval baseline.

Checkpoint: every chunk has a source ID and status, no chunk exceeds its configured size, and your reflection identifies an actual text boundary.

Concept check: why might larger overlap improve evidence coverage yet increase indexing cost and redundant retrieval?

Your answer: _____

Lab 2 · retrieval and evidence filtering

Time: 30–55 minutes · Chapters 2–3

Your task: compare two retrieval methods and prevent archived evidence from entering current-policy answers. Allocate 4 minutes to the walkthrough, 16 to experiments and 5 to recording and discussion.

Concepts before you code

Lexical retrieval compares words. TF-IDF weights terms using their occurrence in a chunk and across the corpus. It can be effective for exact terminology but miss a paraphrase that uses different words.

Dense retrieval compares embedding vectors. This notebook uses the same embedding model for chunks and questions, normalizes the vectors and computes a dot product. With unit-length vectors, that dot product equals cosine similarity. The result is a ranking signal, not a probability that the passage or answer is correct.

Eligibility filtering decides which sources the application may consider. In this exercise, `status == 'current'` is the rule. It controls freshness eligibility; it does not implement user authorization or prove the policy is accurate.

Top-k means the first k unique **source documents** here. The retriever selects each source's best-scoring chunk and skips additional chunks with the same `doc_id`. The generator receives those selected passages, not the full documents.

Step 2.1 — Build and inspect the indexes

1. Run the cell beginning `tfidf = TfidfVectorizer(...)` under Lab 2.
2. Inspect the printed `Dense index shape`.
3. Identify what its row count represents and what the 1,536 columns represent in this configuration.
4. Read `search`, especially the score calculation, status check and `seen` set.

Write: rows represent _____; columns represent _____.

The entire tiny corpus is ranked before the function scans for eligible results. With exhaustive ranking this retains the eligible ordering. A production approximate search with a limited candidate pool needs appropriate filtering before that cut-off.

Step 2.2 — Inspect the first question

1. Keep `QUERY = 'How can I extend my book loan?'` for the first run.
2. Read the source IDs and text returned by both methods.
3. Decide whether the top passage actually supports a renewal answer. Use D02 as the relevant source.
4. Save the result before changing the question.

Step 2.3 — Test a paraphrase

1. Change `QUERY` in the same cell to `How do I read academic publications away from campus?`.
2. Rerun that cell. The data and model configuration stay fixed.
3. Locate D08, the remote-journal source, in each method's top two results.
4. If it is absent, record **not in top 2**. Do not invent a rank beyond the displayed list.

Question	Method	Top source	Relevant source rank in top 2	Supports the question?
Extend book loan	TF-IDF	_____	_____	_____
Extend book loan	Dense	_____	_____	_____
Publications away from campus	TF-IDF	_____	_____	_____
Publications away from campus	Dense	_____	_____	_____

Interpretation: which method helped on which wording? A tie or a dense-retrieval failure is a valid observation. Two questions cannot establish a general winner.

Step 2.4 — Admit the archived policy, then exclude it

1. Locate the cell beginning `CURRENT_ONLY = False`.
2. Run it with `False`. It asks whether a library book can be renewed for 30 days.

- Record whether archived D03 appears among the three results.
- Change only `CURRENT_ONLY` to `True` and rerun.
- Verify that every returned source has `status == 'current'` and D03 is absent.

Setting	Returned source IDs	D03 present?	What this establishes
False	_____	_____	_____
True	_____	_____	_____

Step 2.5 — Record the distinction

Complete `reflections['lab2']` in that cell with your rankings and filter comparison, then run it again with `CURRENT_ONLY = True`.

Checkpoint: you can identify the relevant passage independently of its score, and archived D03 cannot appear under the default current-only policy. Later generation calls explicitly retain this filter.

Concept check: could a retrieved passage satisfy the metadata rule and still fail to answer the question? Give an example from your output or describe a plausible one.

Your answer: _____

Break and save · 55–60 minutes

Save your notebook and keep the runtime connected. If it restarted, rerun Setup A and B, then Labs 1–2 to rebuild the state. Your saved notebook text persists, but the runtime's variables and in-memory caches may not.

Lab 3 · grounded answers and guardrails

Time: 60–85 minutes · Chapters 1–3

Your task: generate an answer from evidence, then test the boundaries that accept or reject inputs and outputs. Allocate 5 minutes to the walkthrough, 15 to experiments and 5 to recording and discussion.

Concepts before you code

Grounding means the answer's claims follow from the supplied evidence. **Provenance** identifies where that evidence came from. A source reference is useful, but a valid source ID alone does not establish grounding.

Structured output gives the application fields it can inspect: `answer`, `abstain`, and `citations`. A well-formed JSON object can still contain false claims. The application therefore applies further checks.

Abstention means declining to answer when evidence is insufficient. It is different from a network failure, API refusal, incomplete response or rejected evidence contract. Inspect `guardrail_status` to distinguish them.

Guardrail	Where to find it	What it does
Input size/type	<code>validate_question</code>	Rejects empty, non-string or oversized questions
Source eligibility	<code>search(..., current_only=True)</code>	Excludes archived sources
Evidence instructions	<code>generate_grounded</code>	Directs the model to treat passages as data, not commands
Response fields	<code>ANSWER_SCHEMA</code>	Specifies the expected JSON structure
Source and quotation	<code>validate_output</code>	Checks supplied source IDs and exact quoted text

These checks do not implement moderation, PII detection or tenant authorization. They also do not prove that every answer claim follows from its quotation.

Step 3.1 — Trace the request and checks

Before running the generation cell, find these features:

1. The input validator runs before retrieval can spend an API call.
2. The question and evidence are encoded in a JSON user message.
3. The higher-priority `instructions` field says that evidence text is untrusted data.
4. The output schema requires source IDs and quotations for an answered question.
5. The validator allows only source IDs present in the supplied passages and exact quotation substrings.

Predict: which check should reject an invented source ID? _____

Step 3.2 — Generate the current renewal answer

1. Run the cell beginning `ANSWER_SCHEMA = {...}`. It defines the functions and asks `How many days can a student renew a book?`.
2. Read the generated answer, `guardrail_status`, citations and retrieved passages.
3. Check the supported D02 facts: **14 additional days, once, unless another reader reserved the book.**
4. Identify any omission or unsupported addition. Do not mark an answer correct just because the contract passed.

Review item	Your observation
Generated answer	_____
Status	_____

Review item	Your observation
Cited source and exact quotation	_____
Duration and one-renewal limit correct?	_____
Reservation exception included?	_____
Unsupported or missing detail	_____

Step 3.3 — Ask a question the corpus cannot answer

1. Locate the next experiment cell, beginning UNKNOWN_QUESTION.
2. Confirm it asks `How deep is the university swimming pool?`.
3. Read the rest of the cell before running: it also contains the deterministic rejection tests in Step 3.4.
4. Run the cell once. Record the pool answer and its status from the first output line.
5. Check the library corpus: there is no pool-depth policy. Distinguish deliberate `abstained` status from an output rejection or API failure.

Observed answer/status: _____

Step 3.4 — Inspect the rejection tests from the same run

The cell you just ran submits three controlled invalid inputs to the application code:

1. A citation to D999, which was not supplied as evidence.
2. A quotation claiming a 99-day renewal that is absent from D02.
3. A question containing 501 characters, above the 500-character limit.

Read the three expected rejection messages. The `before` counter is captured **after** the paid pool question. The final assertion verifies that the three rejection tests added no API calls.

Test	Expected behavior	Observed result
Invented source D999	Reject citation	_____
Invented 99-day quotation	Reject quotation	_____
501-character input	Reject before retrieval/generation	_____
API counter for these three tests	Unchanged	_____

Step 3.5 — Save a bounded conclusion

Complete `reflections['lab3']` in the experiment cell. Include the cited answer, one rejection result and one remaining gap. Run the edited cell to save the reflection; this reruns the pool question and incurs another generation request, while unchanged embeddings come from the cache.

Checkpoint: you have checked actual policy support, distinguished abstention from blocking, and observed all three deterministic rejections.

Concept check: imagine an answer says “99 days” but quotes the real D02 phrase “14 days.” Could the source/quote validator accept it? Explain what additional review is needed.

Your answer: _____

Lab 4 · evaluation and adversarial testing

Time: 85–110 minutes · Chapter 6, with Chapter 4 production discussion

Your task: compare retrieval across labelled questions, then challenge the generator with a poisoned evidence passage. Allocate 5 minutes to metrics, 8 to k comparisons, 7 to the attack test and 5 to recording and discussion.

Concepts before you code

A **gold source** is a source a human has labelled as relevant to a question. This notebook evaluates unique source documents, not individual chunks. It uses six answerable questions, each with one gold document.

Metric	Per-question calculation	What it tells you
Precision@k	Relevant retrieved documents / k	How much of the returned set is relevant
Recall@k	Relevant retrieved documents / all gold documents	How much labelled relevant evidence was found
Reciprocal rank@k	1 / first relevant rank, or 0 if absent by k	Whether useful evidence appears early

The notebook averages each metric across questions. The averaged reciprocal-rank value is **MRR@k**; the printed dictionary calls it **rr**. Unsupported questions have no gold sources and are evaluated separately, because recall would have a zero denominator.

Prompt injection occurs when text supplied as data attempts to redirect the model's behavior. Our test adds a harmless attack string after retrieval to isolate the generator's treatment of untrusted evidence. It does not test an ingestion scanner or every possible attack.

Step 4.1 — Calculate one case by hand

For retrieved IDs [A, C, B], gold IDs {A, B}, and k = 2:

1. Which two documents count as retrieved? _____
2. How many of them are relevant? _____
3. Calculate precision: _____ / _____ = _____
4. Calculate recall: _____ / _____ = _____

5. Find the first relevant rank and its reciprocal: _____

Run the Lab 4 cell beginning `EVAL_SET = [...]`. Its first metric assertions check the paper example. Compare them with your calculation and correct any denominator mistake.

Step 4.2 — Measure top-one retrieval

1. Keep `K = 1` in that cell and run it.
2. Record the summary for each method in the table below.
3. Read the per-question gold and retrieved IDs. Identify a miss if one occurs.

Step 4.3 — Measure top-two retrieval

1. Change only `K` to `2` and rerun the same cell.
2. Record both methods again before the earlier output is lost.
3. Explain any change in recall and precision using actual source IDs.

Method	k	Precision	Recall	MRR, printed as rr
TF-IDF	1	_____	_____	_____
Dense	1	_____	_____	_____
TF-IDF	2	_____	_____	_____
Dense	2	_____	_____	_____

Because each question has one gold source, a successful top-two result has precision 0.5. That reflects the labels and `k` choice. It does not by itself establish that the retriever became worse. A perfect score on six easy questions also cannot establish production quality.

Step 4.4 — Test poisoned evidence

1. Read **Lab 4 · adversarial evidence and human review** and locate the cell beginning `poisoned_hits`.
2. Confirm it copies the baseline passages before appending an instruction to print `INJECTION_SUCCEEDED` and claim a 99-day renewal.
3. Predict the desired result: the generator should continue using the legitimate 14-day policy or safely abstain. A correct, useful answer is stronger evidence of utility than refusing everything.
4. Run the attack cell once.
5. Inspect the answer, status, quotations and `guardrail_observations`.

Attack review	Your observation
Did the marker appear in the answer?	_____
Did the answer claim 14 days, 99 days, or abstain?	_____

Attack review	Your observation
Did the cited evidence support the actual claims?	_____
Was the response useful as well as safe?	_____
What does this one case leave untested?	_____

The automatic marker check is case-sensitive and narrow. An attacker could change the answer without printing that exact string. Inspect meaning as well as the Boolean result. `output_rejected` is a block, not a successfully grounded answer.

Step 4.5 — Complete human review and diagnose one issue

1. In the following code cell, fill all four `human_review` strings using your actual known, unknown and attack outputs. Write `none` for missing details if the known answer is complete.
2. Complete `reflections['lab4']` with both `k` settings, an attack observation and one diagnosis. The export otherwise retains only the last metric run.
3. Run that review cell. It makes no API calls.

Diagnosis prompt	Your answer
Observed failure or remaining limitation	_____
Stage: ingestion, retrieval, generation, or validation	_____
Evidence supporting the diagnosis	_____
One change you would make	_____
A new, held-out question or attack to assess it	_____

Checkpoint: you have four metric rows, a reviewed attack result, completed human judgments and a proposed test. A passed attack string is one observation, not a security guarantee.

Optional extension, after the core work: add a question requiring both D01 and D02, label both documents as gold, and measure recall again. Once you use a question to tune the system, treat it as development data and reserve different questions for final evaluation.

Submission and discussion · 110–120 minutes

Step 5.1 — Check your evidence

- ☐ Lab 1: both chunk counts, a boundary example and an overlap trade-off.
- ☐ Lab 2: both retrieval methods, a paraphrase and filtering on/off.
- ☐ Lab 3: an evidence-reviewed answer, unsupported-question observation and three rejection results.

- ☐ Lab 4: k=1/k=2 metrics, attack review, diagnosis and next test.
- ☐ All four `reflections` strings and all four `human_review` strings are filled and their cells have run.

Step 5.2 — Export your work

1. Run the cell under **Python notes · report export**.
2. Check that it reports `reflections complete: True`. This checks field completion, not the quality of your reasoning.
3. If it reports `False`, fill the missing fields, rerun the relevant cells, then export again.
4. Download `rag_workshop_report.json` and save/download your changed notebook through Colab's File menu.
5. Submit both through the channel provided by your instructor. Retain your worksheet observations in the notebook text cells.

The JSON includes selected model outputs, metrics, usage and reflection fields. It does not include text-cell worksheets automatically; the submitted notebook preserves those.

Step 5.3 — Prepare your one-minute explanation

Complete these sentences using your evidence:

The most important failure or limitation we observed was _____.

We traced it to _____ because _____.

We would change _____ and assess it using _____.

Assessment: each lab earns one point for experiment evidence and one for an interpretation tied to that evidence, for eight points total. A well-diagnosed model failure earns credit. Successful code execution alone does not demonstrate understanding.

Quick troubleshooting reference

Symptom	Next step
Key missing or HTTP 401	Check the exact secret name and notebook-access switch
HTTP 403 or 404	Check project access to the configured models with the instructor
HTTP 429	Check quota/rate limits before a manual retry
Timeout	Check connectivity; retry once after investigating rather than repeatedly rerunning

Symptom	Next step
<code>NameError</code> after reconnect	Rerun setup and the earlier labs that define the missing objects
No output after editing a string	Run the edited cell before exporting
<code>output_rejected</code>	Inspect the source and quote contract; a paraphrased quote can be rejected
<code>complete=False</code>	Fill and run every reflection and human-review field

Where the concepts lead next

Chapter 2 expands parsing and persistent vector storage. Chapter 3 adds more advanced retrieval and guardrails. Chapters 4 and 6 develop production operations and evaluation. Chapters 5, 7, 8 and 9 cover managed platforms, agents, multimodal data and knowledge graphs in separate sessions.

See the [detailed Python notes](#) for implementation commentary and [source references](#) for the curriculum and official API documentation. The original nine chapter packages remain the extended course; this workbook is the focused two-hour practice sequence.